

MOKENGELI

Documentation du projet

IA predictive anti-fraude — Brainsoft 2026

Plateforme d'intelligence artificielle predictive anti-fraude en temps reel. Ce document resume le contexte, l'architecture, le scoring (M1/M2/M3, phishing/vishing), les interfaces et l'API.

MOKENGELI

Documentation du projet

Plateforme d'intelligence artificielle prédictive anti-fraude en temps réel

Équipe Brainsoft · République démocratique du Congo · 2026

1. Présentation

Mokengeli est une plateforme bancaire démonstrative intégrant un moteur d'analyse de risque transactionnel en temps réel. Elle combine :

- une **application client** (virements, comptes, historique) ;
- un **back-office admin** (dashboard, alertes, carte des flux) ;
- une **API** de scoring et de persistance ;
- un assistant conseil **Lisungi** (RAG léger).

L'objectif est de détecter la fraude et l'ingénierie sociale (phishing, vishing, anonymisation réseau) avant le mouvement de fonds, tout en restant compréhensible pour l'analyste et le client.

2. Contexte — fraude numérique en Afrique

L'inclusion financière numérique (mobile money, apps bancaires) progresse rapidement. Les fraudeurs suivent la même dynamique.

Repères (sources publiques)

Source | Élément clé

INTERPOL — Africa Cyberthreat Assessment 2025 | Dans les $\frac{2}{3}$ des pays africains enquêtés, la cybercriminalité représente une part moyenne à élevée des infractions ; **> 30 %** de la criminalité déclarée en Afrique de l'Ouest et de l'Est. Les arnaques en ligne (phishing en tête) sont les plus signalées.

Études régionales (ex. Kenya) | Smishing, phishing et vishing souvent cités autour de **~39 % / 36 % / 33 %** d'exposition — au-dessus des moyennes mondiales.

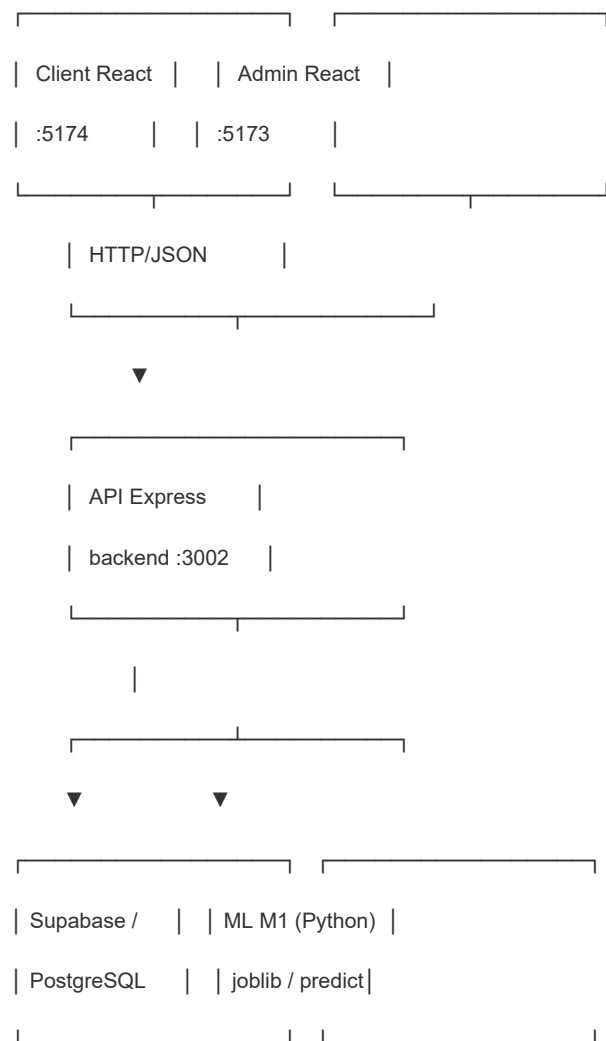
RDC (ENACT / ISS, 2025) | Plus de **56 millions** d'abonnés mobiles ; dépendance au mobile money ; montée du **SIM swap** et de l'ingénierie sociale.

Limites des solutions classiques

- Règles métier figées et listes noires IP ;
- OTP seul (contournable via vishing / SIM swap) ;
- Outils anti-fraude génériques peu adaptés au multi-canal africain ;
- Peu d'explication client et peu de vision analyste unifiée.

Mokengeli se positionne comme une réponse prédictive, multi-signaux et pédagogique.

3. Architecture globale



Dossier | Rôle

`backend/` | API Node.js + Express (runtime principal : `src/index.js`)

`frontend/client/` | Parcours client bancaire

`frontend/admin/` | Back-office analyste

`ml/` | Modèle transactionnel M1 (joblib)

`infra/sql/` | Schéma PostgreSQL

`docs/` | Documentation technique

4. Stack technique

Couche | Technologies

Backend | Node.js ≥ 18, Express 4, Supabase JS, bcryptjs

Client / Admin | React 19, TypeScript, Vite 8, Tailwind 4, React Router 7

Carte admin | MapLibre GL / react-map-gl

Base de données | PostgreSQL (Supabase)

ML | Python + scikit-learn / joblib (régression logistique M1)

5. Fonctionnalités

5.1 Application client

- Connexion (référence client / e-mail / téléphone) ;
- Consultation des comptes et soldes ;
- ****Virement**** avec géolocalisation obligatoire ;
- Création de bénéficiaires (compte bancaire ou mobile money) ;
- Historique des opérations et notifications ;
- ****Lisungi**** : analyse de l'historique et conseils (phishing, vishing, habitudes).

5.2 Back-office admin

- Tableau de bord (volume, fraudes, risque moyen, alertes) ;
- Scores moyens M1 / M2 / M3 / phishing / vishing ;
- Liste des transactions (décision, risque, phishing, vishing) ;
- Modale de détail (scores, motifs, parties) ;
- ****Alertes**** (block / challenge) ;
- ****Carte des flux**** (RDC) ;
- Paramètres (seuils affichés 40 % / 70 %).

6. Pipeline de scoring

Chaque POST /api/v1/transactions/evaluate déclenche :

1. Construction des features (montant, canal, session, réseau, etc.) ;
2. Détection anonymisation (IP serveur + signaux navigateur) : Tor, VPN, proxy, datacenter ;
3. M1 — modèle transactionnel (proba de fraude) ;
4. M2 — anomalie de session ;

5. M3 — comportement / UEBA ;
6. Score combiné = moyenne (M1 + M2 + M3) ;
7. Couche phishing / vishing (heuristique ingénierie sociale) ;
8. Décision + persistance (transactions, scores_evaluation, éventuelle alerte).

Politique de décision (0–100 %)

Score | Décision | Effet

-----: | ----- | -----

0 – 39 % | ****Autoriser**** (`allow`) | Mouvement de solde possible

40 – 69 % | ****OTP**** (`challenge`) | Pas de mouvement ; alerte

70 – 100 % | ****Bloquer**** (`block`) | Pas de mouvement ; alerte critique

Règles dures réseau

- ****Tor**** → blocage ;
- ****VPN**** (ou proxy / hosting typique) → au minimum OTP.

Les motifs (raison codes, niveaux phishing / vishing) sont stockés dans scores_evaluation.texte_motifs (JSON).

7. API principale

Méthode | Route | Description

GET | `/health` | Santé du service

POST | `/api/v1/client/login` | Authentification client

POST | `/api/v1/client/logout` | Déconnexion

GET | `/api/v1/me` | Profil + comptes

GET | `/api/v1/me/notifications` | Notifications

POST | `/api/v1/beneficiaires` | Création bénéficiaire

POST | `/api/v1/transactions/evaluate` | Scoring + persistance

GET | `/api/v1/admin/transactions` | Liste admin (jointures)

GET | `/api/v1/admin/alerts` | Alertes (table + fallback scores)

POST | `/api/v1/lisungi/analyze` | Conseils Lisungi

8. Persistance & alertes

Tables clés (schéma Mokengeli)

- `clients`, `comptes\_bancaires`, `beneficiaires`
- `transactions`
- `scores\_evaluation` (scores M1–M3, combiné, décision, motifs)
- `alerts` (si présente) — sinon dérivation depuis les décisions `challenge` / `block`

Flux décision → alerte

evaluate → décision challenge|block

- upsert scores_evaluation
- insert alerte (best-effort)
- GET /admin/alerts fusionne alerts + scores

9. Lisungi

Assistant conseil client :

1. Charge l'historique récent + scores ;
2. Analyse heuristique (risque moyen, nuit, phishing/vishing, canaux) ;
3. Produit résumé, remarques, conseils, recommandations ;
4. Enrichissement optionnel via OpenAI si clé API configurée.

Objectif : éduquer le client (OTP, appels frauduleux, liens suspects) en s'appuyant sur ses propres opérations.

10. Démarrage local

Prérequis

- Node.js ≥ 18
- Compte Supabase (URL + service role key)
- Python (recommandé pour M1)

Commandes

Variables d'environnement

backend/.env : SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, PORT=3002

npm install

npm --prefix backend run dev

npm --prefix frontend/client run dev

npm --prefix frontend/admin run dev

Service | URL / port

Backend | http://localhost:3002

Client | http://localhost:5174

Admin | http://localhost:5173

Auth démo : AUTH_DISABLED=true possible en local.

11. Déploiement

- **Backend** : Render (Procfile → `src/index.js`, ESM)
- **Frontends** : Vercel (ou équivalent Vite)
- **Données** : Supabase (PostgreSQL)

12. Roadmap indicative

- Remplacer les heuristiques M2/M3 par des modèles entraînés ;
- Enrichir la détection VPN (bases IP commerciales) ;
- Table `alerts` normalisée + workflow analyste (confirmé / faux positif) ;
- Observabilité (métriques latence scoring, taux OTP / block) ;
- Durcissement auth (JWT, rate limiting) en production.

13. Glossaire

Terme | Définition

Phishing | Escroquerie par lien / message visant à dérober des identifiants

Vishing | Escroquerie téléphonique (pression, usurpation banque)

OTP | One-Time Password — code de confirmation

SIM swap | Détournement de numéro mobile pour intercepter SMS / OTP

UEBA | User and Entity Behavior Analytics

14. Références

1. INTERPOL — **Africa Cyberthreat Assessment Report** (2025)
2. ENACT / ISS — SIM swapping en RDC (2025)

3. Documentation interne : README.md, docs/req.md, docs/mpd.md, infra/sql/

Document généré pour le projet Mokengeli — Brainsoft — usage pédagogique et démonstration produit.